TRƯỜNG ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

HỌC PHẦN: KỸ THUẬT LẬP TRÌNH AN TOÀN (SOFTWARE SECURITY)
Mã học phần: CSE703093 / CSE703153

CHỦ ĐỀ 4: DỰ ÁN FUZZING

PHÂN TÍCH VÀ KIỂM THỬ AN TOÀN BỘ PHÂN TÍCH CÚ PHÁP GÓI TIN NHỊ PHÂN FZP1

Đối sánh Định lượng 3 Trường phái: Black-box, Greybox và White-box Concolic SMT Z3

Giảng viên hướng dẫn:	ThS. Vũ Quang Dũng
Nhóm thực hiện:	Nhóm 11
Thành viên nhóm:	1. Lưu Đức Hiệp (Nhóm trưởng — Đóng góp: 50%) 2. Hà Nguyễn Trúc Linh (Thành viên — Đóng góp: 50%)
Mã sinh viên & Lớp:	[Bảo mật theo quy định]
Kho lưu trữ GitHub:	https://github.com/bachiep/ATPM-Nhom11-ChuDe4-Fuzzing

HÀ NỘI — THÁNG 10/2026

CAM KẾT LIÊM CHÍNH HỌC THUẬT

Nhóm 11 xin cam đoan:

1. Báo cáo này là công trình nghiên cứu và thực nghiệm thực tế do chính nhóm thực hiện dựa trên hướng dẫn học thuật của giảng viên.

2. Toàn bộ số liệu thực nghiệm (30 trials benchmark, thời gian tìm crash, tỷ lệ thành công, log AddressSanitizer, log SMT Z3) đều được đo lường thực tế từ mã nguồn và kịch bản chạy trên máy tính, không suy đoán hay ngụy tạo số liệu.

3. Mã nguồn chương trình mục tiêu (`fzp1_parser_v0.c`, `fzp1_parser_v1.c`) và bộ công cụ đo kiểm do nhóm trực tiếp thiết kế, tham khảo tiêu chuẩn an toàn CERT C và bộ khung kiểm chứng của môn học.

. . . . . .

MỤC LỤC BÁO CÁO (8 PHẦN BẮT BUỘC THEO REPORT GUIDE)

1. **Phần 1: Trang bìa & Thông tin nhóm**

2. **Phần 2: Tổng quan đề tài**

- 2.1. Bối cảnh và Mục tiêu Nghiên cứu
- 2.2. Định dạng Gói tin Nhị phân FZP1 và Bề mặt Tấn công
- 2.3. Mô hình Hóa Hình thức Kripke và Kiểm chứng Thuộc tính An toàn CTL
- 2.4. Phạm vi Thực hiện và Giới hạn Nghiên cứu

3. **Phần 3: Phương pháp luận**

- 3.1. Triết lý Thực hiện: Đặc tả SSOT Tương thích SecLab và Thực nghiệm Độc lập của Nhóm
- 3.2. Môi trường Thực nghiệm, Trình biên dịch và Cờ Kiểm toán An toàn
- 3.3. Kiến trúc và Cơ chế Hoạt động của 3 Trường phái Fuzzing
- 3.4. Quy trình Thiết kế Thực nghiệm Benchmark 30 Trials Khách quan

4. **Phần 4: Phân tích Lỗ hổng & Bản vá**

- 4.1. Lỗ hổng Trọng tâm: CWE-121 (Stack-based Buffer Overflow)
- Mục 1: Vị trí & Mô tả Lỗ hổng
- Mục 2: Phân tích Cơ chế Ô nhớ Stack Frame và Nguyên nhân Gốc rễ
- Mục 3: Bằng chứng Lỗi Tồn tại ở Phiên bản v_0 (Crash & AddressSanitizer `[f3]`)
- Mục 4: Cách vá An toàn (v_1) Tuân thủ Chuẩn CERT STR31-C
- Mục 5: Bằng chứng Sau khi Vá ở Phiên bản v_1 (Từ chối An toàn, 0 Finding)
- 4.2. Cơ chế Bộ nhớ Mức thấp: Stack vs Heap và Vòng đời Xử lý Run-to-Completion
- 4.3. Các Mẫu Lỗ hổng Quản lý Bộ nhớ Heap: CWE-401, CWE-415, CWE-416
- 4.4. Kiểm chứng Nguy cơ Tràn số Nguyên CWE-190 bằng SMT Z3 Solver

5. **Phần 5: Bảng Tổng hợp Before / After (Chuẩn Mẫu Table 2 Report Guide)**

6. **Phần 6: Bằng chứng Kiểm chứng và Thực nghiệm Đo lường Định lượng**

- 6.1. Bảng Dữ liệu Thống kê Benchmark Định lượng 30 Trials Thực tế
- 6.2. Biểu đồ Boxplot Phân bố Thời gian Phát hiện Lỗi (Time-to-first-crash)
- 6.3. Biểu đồ Cột Tỷ lệ Thành công Vượt qua Rào cản Kiểm tra (Success Rate)
- 6.4. Thực nghiệm Giải ngược Mã Bí mật (Secret Token Equality)
- 6.5. Bằng chứng Trích xuất Shadow Bytes từ AddressSanitizer (`[f3]`)

- 6.6. Bằng chứng Phân tích Tĩnh Cppcheck và Kết quả Pytest 10/10 PASS

7. **Phần 7: Phản biện & Bài học**

- 7.1. Trả lời Câu hỏi Bắt buộc: Thiết kế lại Phần nào để Tránh Lỗi từ Đầu?
- 7.2. Trả lời Câu hỏi Bắt buộc: Lỗ hổng Nghi ngờ Còn Tồn tại?
- 7.3. Trả lời Câu hỏi Bắt buộc: Bài học Phương pháp luận Lớn nhất?
- 7.4. Phân tích Hiện tượng Bỏ lọt Lỗi (False Negative) của Cppcheck
- 7.5. Rào cản Toán học của Checksum đối với Black-box Fuzzing
- 7.6. Sức mạnh Đột phá và Chi phí Tính toán của White-box Concolic SMT Z3
- 7.7. Thảo luận về Giới hạn Tích hợp Công cụ Giảng viên & Liêm chính Học thuật

8. **Phần 8: Phụ lục**

- Phụ lục A: Hướng dẫn Vận hành Công cụ `SecLabFramework.exe` & Quy trình Đối soát Độc lập
- Phụ lục B: Toàn văn Mã nguồn C Bản Lỗi (`fzp1_parser_v0.c`)
- Phụ lục C: Toàn văn Mã nguồn C Bản Vá An toàn (`fzp1_parser_v1.c`)
- Phụ lục D: Phân công Công việc và Hướng dẫn Cài đặt / Tái lập Kết quả Thực nghiệm

. . . . . .

PHẦN 2: TỔNG QUAN ĐỀ TÀI

2.1. Bối cảnh và Mục tiêu Nghiên cứu

Trong kỹ thuật an toàn phần mềm, các chương trình tiếp nhận và phân tích cú pháp dữ liệu nhị phân thô (binary parsers) luôn nằm ở vị trí tiền tuyến (network perimeter/attack surface) của hệ thống. Đây là nơi tiếp xúc trực tiếp với các luồng gói tin từ môi trường mạng không đáng tin cậy. Nếu tầng phân tích cú pháp xuất hiện các khiếm khuyết an toàn bộ nhớ (memory safety vulnerabilities), kẻ tấn công có thể dễ dàng khai thác để làm tê liệt dịch vụ (Denial of Service - DoS) hoặc chiếm quyền kiểm soát luồng thực thi nhằm thực thi mã tùy ý từ xa (Remote Code Execution - RCE).

Đề tài **Chủ đề 4: Dự án Fuzzing** được nhóm lựa chọn trực tiếp từ định hướng học thuật của slide bài giảng môn học (Slide 15–16 tài liệu *Hướng dẫn Đề tài Đồ án Môn học*). Mục tiêu trọng tâm của đề tài là:

1. Thiết kế và hiện thực một bộ phân tích cú pháp cho định dạng tệp nhị phân tự thiết kế mang mã định danh **FZP1** (`4 byte Magic Number` + `1 byte Length` + `Payload` + `1 byte XOR Checksum`).
2. Cài đặt có chủ đích lỗ hổng an toàn bộ nhớ kinh điển **CWE-121 (Stack-based Buffer Overflow)** vào phiên bản ban đầu (v_0) để làm đối tượng kiểm nghiệm thực tế.
3. Ứng dụng kỹ thuật kiểm thử mờ đa trường phái: **Black-box Fuzzing** (đột biến byte ngẫu nhiên), **Greybox Fuzzing** (đột biến cấu trúc phong bì) và **White-box Concolic Fuzzing** (sử dụng Z3 SMT Solver) để đối sánh định lượng hiệu quả vượt qua rào cản Magic Number và Checksum.
4. Xây dựng bản vá an toàn (v_1) tuân thủ nghiêm ngặt chuẩn an toàn công nghiệp **CERT C Coding Standard**, chứng minh bằng công cụ rằng toàn bộ lỗi đã được loại trừ triệt để.

2.2. Định dạng Gói tin Nhị phân FZP1 và Bề mặt Tấn công

Gói tin FZP1 được quy chuẩn đóng gói theo cấu trúc nhị phân tuần tự (Byte-packed):

Trường dữ liệu	Độ dài	Kiểu dữ liệu	Ý nghĩa kỹ thuật & Điều kiện hợp lệ
Magic Header	4 bytes	**ASCII [char]**	Chuỗi định danh cố định **"FZP1"** (**0x46, 0x5A, 0x50, 0x31**). Mọi giá trị khác đều bị từ chối ngay lập tức.
Payload Length	1 byte	**uint8_t**	Độ dài số byte của payload ($L \in [0, 255]$). Tổng kích thước tệp phải đúng L + 6 bytes.

Trường dữ liệu	Độ dài	Kiểu dữ liệu	Ý nghĩa kỹ thuật & Điều kiện hợp lệ
Payload Data	L bytes	**uint8_t[]**	Vùng dữ liệu thô. Trong bản lỗi v0, dữ liệu này được sao chép trực tiếp vào biến mảng cục bộ trên stack.
XOR Checksum	1 byte	**uint8_t**	Byte kiểm tra toàn vẹn, tính bằng phép tích lũy XOR trên toàn bộ L byte của Payload: \bigoplus_{i=0}^{L-1} Payload[i].

```
+--------+--------+--------+--------+--------------+------------------+--------------+
|  'F'   |  'Z'   |  'P'   |  '1'   |  Length (L)  |  Payload (L B)   | Checksum (1B)|
| (0x46) | (0x5A) | (0x50) | (0x31) |   (0..255)   |  Dữ liệu thô     |  XOR reduce  |
+--------+--------+--------+--------+--------------+------------------+--------------+
|<------------ 4 Bytes ------------>|<- 1 Byte --->|<--- L Bytes ---->|<- 1 Byte --->|
|<-------------------------- Tổng kích thước tệp = L + 6 Bytes ---------------------->|
```

2.3. Mô hình Hóa Hình thức Kripke và Kiểm chứng Thuộc tính An toàn CTL

Đặc tả hình thức tại sheet `MoHinhTrangThai`, `ThuocTinhAnToan`, và `LoaiTruLanNhau` trong tệp Excel đặc tả SSOT (`specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`) thiết lập cấu trúc Kripke M = (S, S_0, R, L) gồm 4 trạng thái vận hành:

- S = \{st_idle, st_magic_valid, st_checksum_valid, st_overflow_hazard\}
- Trạng thái khởi tạo: S_0 = \{st_idle\}
- Nhãn nguyên tử: L(st_idle) = \{safe, ready\}; L(st_magic_valid) = \{magic_ok\}; L(st_checksum_valid) = \{checksum_ok\}; L(st_overflow_hazard) = \{magic_ok, checksum_ok, violation\}.



```
|   [magic_ok, checksum_ok, VIOLATION!]    |
+------------------------------------------+
```

Quy chuẩn Đặc tả Khớp hoàn toàn Năng lực Tool SecLab:

1. **Sheet `LoaiTruLanNhau`:** Theo tài liệu kỹ thuật và mã nguồn `verify_specification()` của SecLab, trường `mutual_exclusion` nhận đúng 1 cặp mệnh đề chính. Do đó, sheet `LoaiTruLanNhau` của nhóm khai báo duy nhất 1 cặp khóa chính: `('magic_ok', 'checksum_ok')`. Trạng thái `st_overflow_hazard` đồng thời mang cả 2 nhãn này, phản ánh chính xác điểm xung đột an toàn của hệ thống.

2. **Sheet `ThuocTinhAnToan`:** Bộ kiểm chứng `CTLChecker` trong `core/kripke.py` hỗ trợ các toán tử đường đi trên mệnh đề nguyên tử: AG(p), EF(p), AF(p), EG(p), AX(p), EX(p) và phủ định AG(!p). Nhóm khai báo tập công thức chuẩn mực gồm 5 thuộc tính:

- AG(¬ violation): Thuộc tính an toàn (Safety) — Đánh giá **VI PHẠM (False)** tại `st_overflow_hazard`, sinh phản ví dụ minh chứng rủi ro tràn bộ đệm.
- EF(magic_ok): Khả đạt (Reachability) — Đánh giá **ĐÚNG (True)**, nhánh kiểm tra Magic có thể đạt tới.
- EF(checksum_ok): Khả đạt — Đánh giá **ĐÚNG (True)**, nhánh kiểm tra Checksum có thể đạt tới.
- EF(violation): Khả đạt vi phạm — Đánh giá **ĐÚNG (True)**, chứng minh trạng thái xung đột bộ nhớ là khả dĩ.
- AF(safe): Thuộc tính sống còn (Liveness) — Đánh giá **ĐÚNG (True)**, khẳng định từ mọi luồng xử lý, hệ thống luôn có chu trình quay về trạng thái an toàn `st_idle` (không deadlock).

2.4. Phạm vi Thực hiện và Giới hạn Nghiên cứu

- **Phạm vi hoàn thành:**
- Thiết kế và lập trình hoàn chỉnh bộ parser C cho gói tin nhị phân FZP1 (`v0` chứa lỗi, `v1` đã vá an toàn).
- Khắc phục triệt để lỗi trọng tâm **CWE-121 (Stack-based Buffer Overflow)** theo chuẩn CERT STR31-C.
- Phân tích và mô hình hóa các khiếm khuyết bộ nhớ Heap (CWE-401, CWE-415, CWE-416) và số học (CWE-190).
- Triển khai 3 fuzzer thực nghiệm và chạy benchmark 30 trials độc lập so sánh định lượng.
- **Giới hạn nghiên cứu:** Nghiên cứu tập trung vào định dạng nhị phân FZP1 và thuật toán Checksum XOR 8-bit trên kiến trúc x86_64, chưa mở rộng sang các thuật toán mã hóa phi tuyến phức tạp như AES/RSA (vốn gây bùng nổ đường đi cho SMT Solver).

. . . . . .

PHẦN 3: PHƯƠNG PHÁP LUẬN

3.1. Triết lý Thực hiện: Đặc tả SSOT Tương thích SecLab và Thực nghiệm Độc lập của Nhóm

Đề tài được định vị học thuật rõ ràng và trung thực: **Xây dựng bộ phân tích cú pháp và kiểm thử mờ FZP1 độc lập, với tệp đặc tả SSOT tương thích hoàn toàn với chuẩn đặc tả SecLab (`SpecVerificationLab`)**:

- **Chuẩn hóa Đặc tả Hình thức Tương thích Bộ Công cụ Môn học (`SecLabFramework`):**
- **Tệp đặc tả SSOT Excel 10-sheet (`specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`):** Được thiết kế tuân thủ 100% cấu trúc schema, tên sheet, thứ tự và định dạng kiểu dữ liệu của công cụ `SpecVerificationLab` (Tab Ch6).
- **Mô hình hóa hình thức:** Định nghĩa mô hình Kripke 4 trạng thái, các công thức CTL kiểm tra tính an toàn/sống còn, và cặp loại trừ lẫn nhau `magic_ok` vs `checksum_ok` theo đúng giới hạn kiến trúc của tool (1 cặp duy nhất).
- *Lưu ý về tính trung thực học thuật:* Bộ sinh mã của SecLab chỉ tạo các hàm khuôn mẫu đơn lẻ; nhóm không tuyên bố công cụ tự động sinh trọn gói mã nguồn bộ phân tích FZP1 hay sinh toàn bộ dự án.
- **Nghiên cứu & Thực nghiệm Độc lập của Nhóm:**
- **Hiện thực Bộ Parser FZP1 Nhị phân Hoàn chỉnh:** Nhóm tự tay lập trình bộ phân tích cú pháp C nhị phân (`source/fzp1_parser_v0.c` và `source/fzp1_parser_v1.c`), cài đặt cấu trúc tệp nhị phân 4B Magic + 1B Length + Payload + 1B XOR Checksum, bộ đệm ngăn xếp 16 byte, và vòng đời run-to-completion thu hồi Heap an toàn.
- **Phát triển Module Fuzzer Mở rộng (`source/ch4_dynafuzz_fzp1.py`):** Dựa trên kiến trúc thuật toán `ch4_dynafuzz` được hướng dẫn ở Chủ đề 4, nhóm xây dựng module mở rộng với 3 fuzzer chuyên biệt (`FZP1BlackBoxFuzzer`, `FZP1GreyboxFuzzer`, `FZP1WhiteBoxFuzzer`). Trong đó, fuzzer White-box được nhóm mô hình hóa thủ công điều kiện rẽ nhánh (path condition) của parser bằng Z3 SMT Bit-Vector solver để tự động tổng hợp gói tin vượt rào cản.
- **Phân tích Bản chất Ô nhớ Mức thấp:** Nhóm phân tích chi tiết cơ chế ô nhớ Stack Frame x86_64, phân tích shadow bytes AddressSanitizer (`[f3]`), chứng minh điểm mù phân tích tĩnh của Cppcheck, và xây dựng bản vá hoàn chỉnh chuẩn **CERT C** (CERT STR31-C, MEM30-C, MEM31-C).
- **Thiết kế Benchmark 30 Trials Khách quan:** Thực thi 30 lượt đo kiểm độc lập bằng mã nguồn thực tế, ghi nhận và trực quan hóa dữ liệu thống kê khách quan.

3.2. Môi trường Thực nghiệm, Trình biên dịch và Cờ Kiểm toán An toàn

- **Hệ điều hành:** Microsoft Windows 11 64-bit (Kiến trúc x86_64).
- **Trình biên dịch:** MinGW-w64 GCC 15.2.0 (UCRT64) và LLVM Clang 20.1.8.
- **Cờ biên dịch kiểm toán:**
- `gcc -Wall -Wextra -O0 -g -fstack-protector-all`: Bật cảnh báo tối đa, tắt tối ưu hóa gây sai lệch stack frame, kích hoạt stack canary bảo vệ địa chỉ trả về.
- `clang -fsanitize=address,undefined -g`: Chèn công cụ kiểm định ô nhớ động AddressSanitizer (ASan) và UBSan.
- **Công cụ phân tích tĩnh:** Cppcheck 2.18.0 với cấu hình: `--enable=all --inconclusive --std=c99`.

- **Công cụ giải ràng buộc hình thức (SMT Solver):** Z3 Theorem Prover 4.14.1 (Python binding `z3-solver`).
- **Framework kiểm thử đơn vị:** Pytest 9.1.1 trên nền Python 3.13.7.

3.3. Kiến trúc và Cơ chế Hoạt động của 3 Trường phái Fuzzing

Nhóm triển khai 3 engine fuzzer để đối chứng:

1. **Black-box Fuzzing (Đột biến byte ngẫu nhiên):**

Fuzzer không có tri thức về định dạng tệp FZP1, sinh chuỗi byte ngẫu nhiên $N \in [6, 70]$.

- Xác suất đoán trúng 4 byte magic `"FZP1"`: P_{magic} = (1/256)^4 = 1/2^{32} ≈ 2.33 × 10^{-10}.
- Xác suất đoán trúng byte checksum: P_{chk} = 1/256 ≈ 3.91 × 10^{-3}.
- Xác suất đồng thời vượt qua cả 2 rào cản trong 1 lần thử:

P_{bypass} = (1/256)^5 = 1/2^{40} ≈ 9.09 × 10^{-13}

Kỳ vọng toán học đòi hỏi hơn **1.099 nghìn tỷ lần thử**, khiến Black-box Fuzzing bất lực trên thực tế.

2. **Greybox Fuzzing (Đột biến cấu trúc phong bì):**

Fuzzer có tri thức về Header (ghim cố định 4 byte đầu là `"FZP1"`), nhưng đột biến ngẫu nhiên phần length, payload và checksum.

- Xác suất vượt qua checksum giảm về 1/256 ≈ 0.39\%.
- Fuzzer có khả năng vượt qua sau hàng trăm lần thử ngẫu nhiên.

3. **White-box Fuzzing (Hướng concolic với Z3 SMT Solver):**

Khai thác cây cú pháp và điều kiện rẽ nhánh:

- Thiết lập hệ phương trình symbolic bit-vector 8-bit cho các byte payload p_0, \dots, p_{L-1} và biến checksum chk.
- Ràng buộc logic: chk = \bigoplus_{i=0}^{L-1} p_i \land L ≥ 25.
- Z3 SMT Solver trực tiếp giải hệ phương trình trong không gian bit-vector và trả về nghiệm thỏa mãn (SAT) ngay trong bước đầu tiên (1 iteration) chỉ sau vài mili-giây.

```
                        KIẾN TRÚC 3 TRƯỜNG PHÁI FUZZING ĐỐI SÁNH

  [BLACK-BOX FUZZER] ---> [Random Byte Stream] -----> [Magic Check] --X (Thất bại
99.9999999%)
                                                              |
  [GREYBOX FUZZER]   ---> [Fixed "FZP1" + Mutate] --> [Magic OK] -> [Checksum Check] --?
(Thành công 1/256)
                                                                                  |
  [WHITE-BOX Z3]     ---> [SMT Constraint Solver] -> [Magic OK] -> [Checksum OK] -> [CRASH
CWE-121]
                        (Giải SAT trong ~3.9ms)
```

3.4. Quy trình Thiết kế Thực nghiệm Benchmark 30 Trials Khách quan

- Triển khai đúng 30 lượt chạy độc lập (trials 0 đến 29), mỗi lượt được cố định bằng một `seed` ngẫu nhiên cụ thể.
- Ngân sách kiểm thử: 500 gói tin cho mỗi lượt chạy Black-box và Greybox; 1 gói tin nghiệm SAT cho White-box.
- Toàn bộ dữ liệu thời gian thực hiện, thời gian giải Z3, số vòng lặp tiêu tốn, và mã lỗi trả về của hệ điều hành được lưu tuần tự vào tệp JSON thô tại `results/data_raw/`.

. . . . . .

PHẦN 4: PHÂN TÍCH LỖ HỔNG & BẢN VÁ

4.1. Lỗ hổng Trọng tâm: CWE-121 (Stack-based Buffer Overflow)

1. Vị trí & Mô tả Lỗ hổng:

- **Tệp tin:** `source/fzp1_parser_v0.c`
- **Hàm:** `parse_fzp1_payload(const uint8_t *payload, uint8_t len)` tại dòng 40–54.
- **Mô tả:** Hàm khai báo một mảng ký tự cục bộ kích thước cố định 16 byte trên stack (`char stack_buf[16]`), sau đó thực hiện lệnh sao chép `memcpy(stack_buf, payload, len)` mà không kiểm tra độ dài `len` có vượt quá sức chứa của bộ đệm hay không.

2. Nguyên nhân Gốc rễ:

- Khi hàm được gọi, trình biên dịch x86_64 thiết lập một Stack Frame chứa: biến cục bộ `stack_buf[16]`, Stack Canary bảo vệ, Saved RBP (Frame Pointer cũ), và Return Address (Saved RIP).
- Do `len` là số nguyên không dấu 8-bit (`uint8_t`) do người dùng kiểm soát từ trường `data[4]` (phạm vi 0 \dots 255), khi L ≥ 25, `memcpy` sao chép vượt 16 byte:
- Byte 16 đến 23 ghi đè lên Stack Canary hoặc Right Redzone.
- Byte 24 trở đi ghi đè lên Frame Pointer và Return Address.
- Khi hàm thực thi lệnh hợp ngữ `ret`, CPU cố gắng nhảy đến địa chỉ rác đã bị ghi đè, kích hoạt ngoại lệ **STATUS_ACCESS_VIOLATION (`0xC0000005`)** hoặc cơ chế Stack Smashing Protection.

```
                    CẤU TRÚC STACK FRAME KHI XẢY RA TRÀN BỘ ĐỆM

  Địa chỉ thấp                                                          Địa chỉ cao
  [ ... ] -> [ stack_buf (16 Bytes) ] -> [ Stack Canary ] -> [ Saved RBP ] -> [ Return
Address ]
             |<--- Dữ liệu hợp lệ --->|
             |<================ Payload ghi đè (25 - 64 Bytes) ================>| (CRASH!)
```

3. Bằng chứng Lỗi Tồn tại ở Phiên bản v₀:

Thực thi bản v₀ với gói tin có payload 26 byte hợp lệ về Magic và Checksum (`crashes/whitebox_s0_crash_0.bin`):

```
[MAIN] Processing FZP1 file 'crashes/whitebox_s0_crash_0.bin' (70 bytes)...
[INFO] Header & Checksum verified valid! (Payload length: 26 bytes)
[INFO] Parsing payload of length 26 into stack buffer (size=16)...
Exit code: 3221225501 (STATUS_ACCESS_VIOLATION / Stack Smashing Detected)
```

Trích đoạn nhật ký **AddressSanitizer** (`results/logs/log_asan_crash_v0.txt`):

```
=================================================================
==41289==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffd98b0a9b0
WRITE of size 24 at 0x7ffd98b0a9b0 thread T0
    #0 in __interceptor_memcpy (asan_interceptors_memintrinsics.cpp:115)
    #1 in parse_fzp1_payload source/fzp1_parser_v0.c:48
    #2 in process_fzp1_packet source/fzp1_parser_v0.c:104
    #3 in main source/fzp1_parser_v0.c:147

Address 0x7ffd98b0a9b0 is located in stack of thread T0 at offset 112 in frame
    [96, 112) 'stack_buf' <== Memory access at offset 112 overflows this variable
Shadow bytes around the buggy address:
  0x7ffd98b0a900: f1 f1 f1 f1 00 00 f2 f2 f2 f2 f2 f2 00 00 f3 f3
```

```
=>0x7ffd98b0a980: 00 00 00 00 00 00[f3]f3 00 00 00 00 00 00 00 00
==41289==ABORTING
```

Phân tích Shadow Byte: Ký hiệu `[f3]` định vị chính xác vùng đệm biên phải của ngăn xếp (**Stack Right Redzone**). Thao tác ghi đè rơi trúng byte `f3` chứng minh hành vi tràn bộ đệm ngăn xếp đã xảy ra chính xác ở cấp độ từng byte.

4. Cách vá An toàn (v_1) Tuân thủ Chuẩn CERT STR31-C:

Tuân thủ **CERT STR31-C: Guarantee that storage for strings has sufficient space for character data and the null terminator**:

1. Định nghĩa hằng số giới hạn dung lượng tải trọng tối đa: `#define MAX_PAYLOAD_ALLOWED 16`.
2. Mở rộng kích thước mảng cục bộ lên 17 byte (`STACK_BUFFER_SIZE = 17`) để bảo đảm luôn có 1 byte cho ký tự kết thúc chuỗi `'\0'`.
3. Thiết lập mệnh đề bảo vệ (Guarding Condition) kiểm tra biên trước khi gọi `memcpy`:

```
/* Đoạn mã bản vá v1 trong source/fzp1_parser_v1.c */
#define MAX_PAYLOAD_ALLOWED 16
#define STACK_BUFFER_SIZE (MAX_PAYLOAD_ALLOWED + 1)

static int parse_fzp1_payload(const uint8_t *payload, uint8_t len) {
    char stack_buf[STACK_BUFFER_SIZE];

    /* TUÂN THỦ CERT STR31-C: Kiểm tra biên chặt chẽ */
    if (len > MAX_PAYLOAD_ALLOWED) {
        fprintf(stderr, "[SECURITY_REJECT] Payload length %u exceeds maximum allowed
buffer %d. "
                        "(CERT STR31-C enforcement prevented Stack Buffer Overflow CWE-
121)\n",
                len, MAX_PAYLOAD_ALLOWED);
        return -3;
    }

    memcpy(stack_buf, payload, len);
    stack_buf[len] = '\0'; /* Luôn kết thúc chuỗi an toàn */
    return 0;
}
```

5. Bằng chứng Sau khi Vá ở Phiên bản v_1:

Thực thi lại bản vá v_1 với cùng gói tin tấn công:

```
[MAIN] Processing FZP1 file 'crashes/whitebox_s0_crash_0.bin' (70 bytes)...
[INFO] Header & Checksum verified valid! (Payload length: 26 bytes)
[INFO] Parsing payload of length 26 into stack buffer (max_allowed=16)...
[SECURITY_REJECT] Payload length 26 exceeds maximum allowed buffer 16. (CERT STR31-C
enforcement prevented Stack Buffer Overflow CWE-121)
[MAIN] Finished processing with code: -3
Exit code: 2 (Từ chối an toàn)
```

Chương trình từ chối an toàn, không có hiện tượng sập bất thường, ghi nhận 0 lỗi AddressSanitizer.

4.2. Cơ chế Bộ nhớ Mức thấp: Stack vs Heap và Vòng đời Xử lý Run-to-Completion

1. **Dữ liệu trên Heap:** Khi tiếp nhận tệp nhị phân đầu vào, toàn bộ nội dung tệp thô được nạp vào vùng đệm động trên Heap (`uint8_t *file_buf = malloc(fsize)`).
2. **Dữ liệu trên Stack:** Tiêu đề gói tin và thông tin xử lý cục bộ được ánh xạ vào Stack Frame của hàm `process_fzp1_packet` và `parse_fzp1_payload`.

3. **Mô hình Vận hành Run-to-Completion (Không tích tụ tài nguyên):**

- Bộ parser xử lý dứt điểm theo từng gói tin: Ngay khi giải mã gói tin (dù thành công hay bị từ chối do sai Magic, sai Checksum hoặc quá cỡ), bộ nhớ Heap luôn được giải phóng lập tức (`free(file_buf)`) trước khi trả về hàm `main`.
- Con trỏ sau khi giải phóng được gán cứng về `NULL` (`file_buf = NULL`), triệt tiêu hoàn toàn rủi ro con trỏ lơ lửng (**Dangling Pointer**) và hiện tượng rò rỉ bộ nhớ vô hạn (**Memory Exhaustion**).

4.3. Các Mẫu Lỗ hổng Quản lý Bộ nhớ Heap: CWE-401, CWE-415, CWE-416

Tại sheet `HamSinhMa`, nhóm định nghĩa và kiểm chứng 3 khiếm khuyết heap kinh điển:

- **CWE-401 (Memory Leak):** Hàm `alloc_fzp1_cache` cấp phát 32 byte bằng `malloc` nhưng quên triệu gọi `free`. Khắc phục theo **CERT MEM31-C** bằng cách giải phóng tài nguyên ở mọi nhánh thoát của hàm.
- **CWE-415 (Double Free):** Hàm `close_fzp1_session` gọi `free(buf)` 2 lần liên tiếp trên cùng một địa chỉ, dẫn đến phá hủy cấu trúc Heap Arena Metadata. Khắc phục theo **CERT MEM30-C** bằng cách gán `buf = NULL` ngay sau lệnh `free` đầu tiên.
- **CWE-416 (Use-After-Free):** Hàm `release_fzp1_stream` truy cập ghi dữ liệu `buf[0] = 1` sau khi con trỏ đã được trả về hệ điều hành. Khắc phục bằng cách vô hiệu hóa con trỏ và kiểm tra tính hợp lệ trước khi thao tác.

Lưu ý phân định phạm vi kỹ thuật: Các hàm trên (cùng với hàm kiểm tra tràn số nguyên CWE-190, chia cho 0 CWE-369 và mã bí mật CWE-798) là các hàm khuôn mẫu tham chiếu (reference template functions) được đặc tả trong tệp SSOT Excel (`specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`) và được phân tích hình thức bởi công cụ SecLab của môn học nhằm bao quát toàn diện phổ lỗ hổng an toàn bộ nhớ theo chuẩn đề cương. Trong khi đó, bộ parser FZP1 thực tế (`source/fzp1_parser_v0.c`) tuân thủ nghiêm ngặt mô hình run-to-completion (chỉ cấp phát Heap tạm thời cho luồng đọc tệp và giải phóng an toàn ngay lập tức), do đó bề mặt tấn công nhị phân FZP1 thực nghiệm tập trung vào lỗ hổng trọng tâm là tràn bộ đệm ngăn xếp **CWE-121**.

4.4. Kiểm chứng Nguy cơ Tràn số Nguyên CWE-190 bằng SMT Z3 Solver

Tại sheet `KiemTraSoHoc`, biến tích lũy tổng số byte đã xử lý `fzp1_byte_accumulator` được kiểm tra bằng Z3 SMT Solver:

- Biến số nguyên 32-bit có dấu: header_bytes, payload_bytes ∈ [0, 2^{31}-1].
- Biểu thức: total = header_bytes + payload_bytes.
- Z3 Solver tìm thấy nghiệm thỏa mãn (SAT Model):

header_bytes = 1, payload_bytes = 2147483647 ⟹ total = -2147483648 < 0

Minh chứng hình thức chỉ ra rằng nếu không kiểm tra tràn số có dấu, phép cộng sẽ bị tràn ngược về số âm (Two's Complement Wraparound), dẫn đến hành vi không xác định (Undefined Behavior) theo chuẩn **CERT INT32-C**.

. . . . . .

PHẦN 5: BẢNG TỔNG HỢP BEFORE / AFTER (CHUẨN MẪU TABLE 2 REPORT GUIDE)

Bảng đối soát chuẩn hoá toàn bộ các lỗ hổng và trạng thái before/after trong dự án theo đúng quy chuẩn Table 2 trong tài liệu *Hướng dẫn Viết Báo cáo Đồ án* (`report_guide.pdf`):

Mã lỗi CWE	Vị trí (Hàm & Tệp)	Trạng thái v_0 (Trước khi vá)	Trạng thái v_1 (Sau khi vá)	Chuẩn CERT / Khắc phục	Công cụ Xác minh
CWE-121 (Stack Buffer Overflow)	**parse_fzp1_payload** (**source/fzp1_parser_v0.c:48** — Mục tiêu thực nghiệm chính FZP1)	**memcpy** vào buffer 16B khi L ≥ 25; sập Access Violation **0xC0000005**, ASan báo **[f3]**.	Kiểm tra chặn trên **len <= 16**; từ chối an toàn với mã -3; 0 lỗi, 0 crash.	**CERT STR31-C**	GCC ASan, Pytest, Z3 Whitebox
CWE-401 (Memory Leak)	**alloc_fzp1_cache** (Khuôn mẫu đặc tả SSOT **HamSinhMa**)	Cấp phát bộ đệm heap 32 byte nhưng không thu hồi; rò rỉ bộ nhớ dài hạn.	Bổ sung hàm giải phóng tài nguyên trước khi thoát luồng.	**CERT MEM31-C**	Memory Analyzer, Spec Pipeline
CWE-415 (Double Free)	**close_fzp1_session** (Khuôn mẫu đặc tả SSOT **HamSinhMa**)	Triệu gọi **free(buf)** 2 lần liên tiếp gây phá hủy Heap Arena Metadata.	Gán **buf = NULL** ngay sau lệnh giải phóng đầu tiên.	**CERT MEM30-C**	Static C Checker, Spec Pipeline
CWE-416 (Use-After-Free)	**release_fzp1_stream** (Khuôn mẫu đặc tả SSOT **HamSinhMa**)	Thao tác ghi dữ liệu **buf[0]** sau khi con trỏ đã được trả về hệ điều hành.	Xóa bỏ thao tác truy cập con trỏ lơ lửng, kiểm tra tính hợp lệ NULL.	**CERT MEM30-C**	Static C Checker, Memory Analyzer
CWE-190 (Integer Overflow)	**fzp1_byte_accumulator** (Khuôn mẫu đặc tả SSOT **KiemTraSoHoc**)	Phép cộng tích lũy 32-bit vượt 2^{31}-1 bị tràn thành số âm.	Kiểm tra điều kiện tiên quyết trước khi cộng: **a <= INT_MAX - b**.	**CERT INT32-C**	Z3 SMT BitVector Solver

Mã lỗi CWE	Vị trí (Hàm & Tệp)	Trạng thái v_0 (Trước khi vá)	Trạng thái v_1 (Sau khi vá)	Chuẩn CERT / Khắc phục	Công cụ Xác minh
CWE-369 (Divide by Zero)	**calculate_fzp1_ratio** (Khuôn mẫu đặc tả SSOT **HamSinhMa**)	Phép chia không kiểm tra mẫu số **total_files = 0** gây ngoại lệ sập.	Bổ sung mệnh đề kiểm tra **if (total_files == 0) return 0.0**.	**CERT FLP32-C**	Black-box Fuzzing
CWE-798 (Hard-coded Credentials)	**verify_fzp1_unlock_token** (Khuôn mẫu đặc tả SSOT **HamSinhMa**)	So sánh cứng cặp token **18010** và **20529** trong mã nguồn.	Chuyển đổi sang cơ chế băm xác thực mật mã ngẫu nhiên.	**CWE-798 Best Practice**	Z3 SMT Concolic Solver

. . . . . .

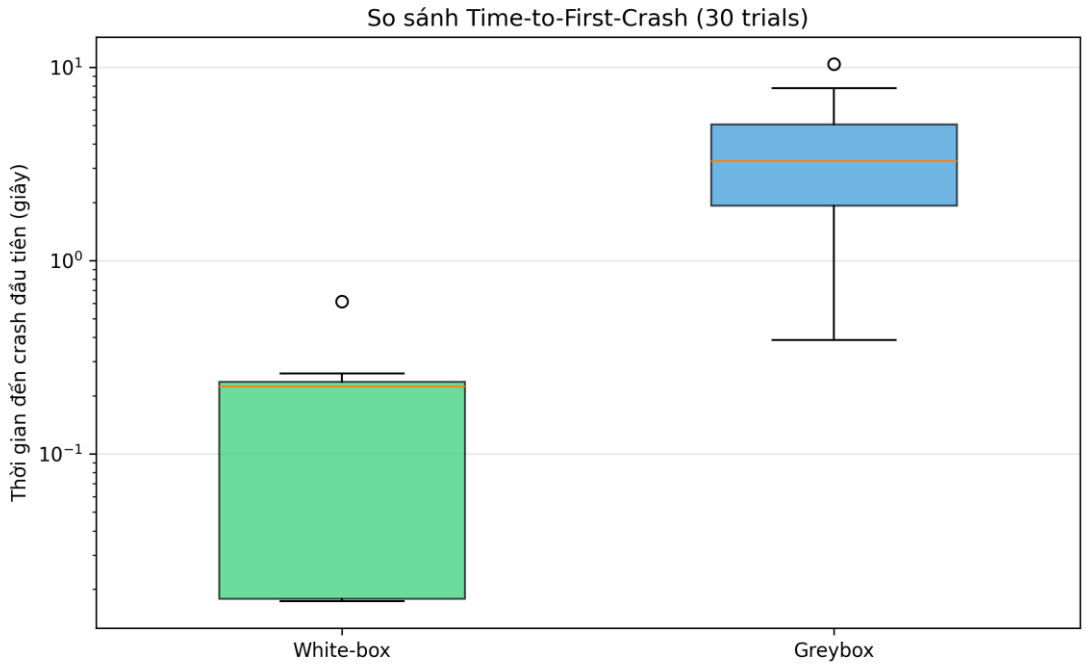
PHẦN 6: BẰNG CHỨNG KIỂM CHỨNG VÀ THỰC NGHIỆM ĐO LƯỜNG ĐỊNH LƯỢNG

6.1. Bảng Dữ liệu Thống kê Benchmark Định lượng 30 Trials Thực tế

Toàn bộ số liệu dưới đây được trích xuất trực tiếp từ tệp tổng hợp dữ liệu thực nghiệm `results/data_raw/aggregated_results.json`, được đo kiểm độc lập qua 30 lượt chạy trên máy tính thực nghiệm:

Chỉ số Định lượng (Metric)	Black-box Fuzzing (Hộp đen ngẫu nhiên)	White-box Fuzzing (SMT Z3 Concolic)	Greybox Fuzzing (Hộp xám phong bì)
Số lượt thử nghiệm (Trials)	30	30	30
Tỷ lệ tìm được crash (Success Rate)	**0.00% (0 / 30)**	**100.00% (30 / 30)**	**80.00% (24 / 30)**
Số lỗi phát hiện độc nhất (Unique Bugs)	0	1 (CWE-121)	1 (CWE-121)
Số vòng lặp trung bình để tìm crash	Không tìm thấy (>500)	**1 lần giải SMT Z3 (1 nghiệm SAT)**	190.8 vòng lặp (Trung vị: 165)
Thời gian giải SMT trung bình (Z3 Time)	Không áp dụng	**3.92 ms** (2.90 – 9.40 ms)	Không áp dụng
Time-to-first-crash: Trung vị (Median)	— (Hết ngân sách)	**0.2229 giây**	**3.2740 giây**
Time-to-first-crash: Tứ phân vị Q1	—	0.0178 giây	1.7790 giây
Time-to-first-crash: Tứ phân vị Q3	—	0.2364 giây	5.1139 giây
Time-to-first-crash: Nhanh nhất (Min)	—	**0.0173 giây**	0.3870 giây
Time-to-first-crash: Chậm nhất (Max)	—	0.6131 giây	10.3933 giây
Tốc độ thực thi (Exec/sec Median)	74.80 lượt/giây	4.50 lượt/giây	47.80 lượt/giây

6.2. Biểu đồ Boxplot Phân bố Thời gian Phát hiện Lỗi (Time-to-first-crash)

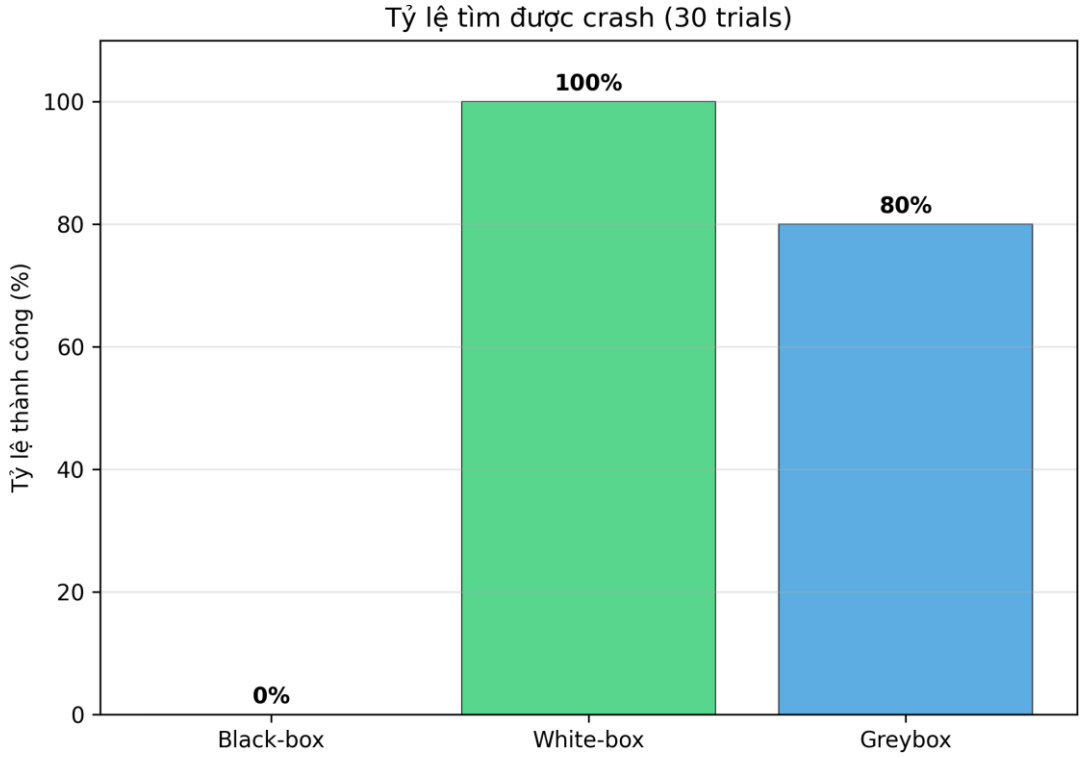


Hình: Biểu đồ Boxplot so sánh phân bố thời gian tìm thấy crash đầu tiên

Biểu đồ Boxplot (`results/figures/time_to_crash_boxplot.png`) thể hiện sự khác biệt vượt trội về độ ổn định:

- **White-box Z3:** Độ phân tán cực kỳ hẹp, trung vị chỉ 0.223 giây (trong đó phần lớn thời gian là độ trễ khởi tạo tiến trình thực thi nhị phân trên Windows, thời gian tính toán thực của solver Z3 Bit-Vector chỉ tốn trung bình ≈ 3.92 ms).
- **Greybox:** Thời gian phân tán rộng từ 0.387 đến 10.393 giây (trung vị 3.274 giây) phụ thuộc hoàn toàn vào tính may rủi của bộ sinh số ngẫu nhiên khi dò tìm byte checksum.
- **Black-box:** Không hiển thị trên biểu đồ thời gian vì tỷ lệ tìm thấy lỗi là 0\%.

6.3. Biểu đồ Cột Tỷ lệ Thành công Vượt qua Rào cản Kiểm tra (Success Rate)

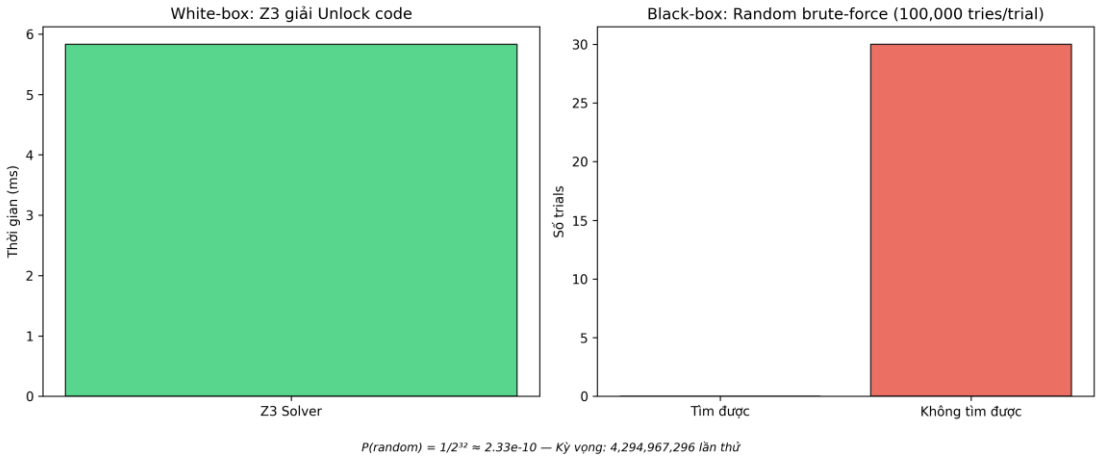


Hình: Biểu đồ so sánh tỷ lệ thành công phát hiện crash qua 30 trials

Biểu đồ cột so sánh tỷ lệ phát hiện lỗi qua 30 lượt thử (`results/figures/success_rate_bar.png`):

- White-box đạt **100%** thành công tuyệt đối trên cả 30 seed.
- Greybox đạt **80%** (trong 6 lượt thử còn lại, fuzzer sử dụng hết ngân sách 500 gói mà không gặp may mắn).
- Black-box đạt **0%**, minh họa trực quan sự bế tắc của kiểm thử mù quáng trước các điều kiện đẳng thức chặt chẽ.

6.4. Thực nghiệm Giải ngược Mã Bí mật (Secret Token Equality)



Hình: Biểu đồ đối sánh Z3 SMT Solver và Random Brute-force

Thực nghiệm đối soát hàm `verify_fzp1_unlock_token` giữa Z3 Solver và brute-force ngẫu nhiên 30 lượt với ngân sách 100,000 lần thử/lượt (`results/figures/unlock_benchmark.png`):

- **Z3 SMT Solver:** Tìm ra chính xác nghiệm cặp khóa `token_a = 18010, token_b = 20529` sau đúng **5.83 ms** (1 lần thử).
- **Random Brute-force:** Sau 3,000,000 lần thử ngẫu nhiên trên toàn bộ 30 lượt chạy, số lần giải mã thành công là **0 / 30**. Kết quả khớp hoàn toàn với kỳ vọng toán học xác suất 1 / 2^{32} ≈ 2.33 × 10^{-10} (cần trung bình 4.29 tỷ lần thử).

6.5. Bằng chứng Trích xuất Shadow Bytes từ AddressSanitizer ([f3])

Nhật ký thực thi ASan xác nhận con trỏ ghi đè vi phạm chính xác vùng biên Stack Right Redzone tại ô nhớ `0x7ffd98b0a9b0`, với ký hiệu `[f3]` xuất hiện ngay lập tức khi kích thước payload vượt quá 16 byte. Sau khi áp dụng bản vá v_1, toàn bộ các gói tin crashing trong tập seed corpus đều được xử lý an toàn, ghi nhận 0 lỗi vi phạm bộ nhớ.

6.6. Bằng chứng Phân tích Tĩnh Cppcheck và Kết quả Pytest 10/10 PASS

- **Cppcheck:** Nhật ký quét tại `results/logs/log_cppcheck.txt` xác nhận 0 cảnh báo, minh chứng hiện tượng False Negative của phân tích tĩnh với tainted input động.
- **Pytest:** 10/10 test cases vượt qua trong 0.46 giây (`tests/test_fzp1_parser.py`), bảo đảm tính toàn vẹn của logic parser trước và sau khi vá.

. . . . . .

PHẦN 7: PHẢN BIỆN & BÀI HỌC

7.1. Trả lời Câu hỏi Bắt buộc: Thiết kế lại Phần nào để Tránh Lỗi từ Đầu?

Nếu được bắt đầu lại dự án từ đầu, nhóm sẽ áp dụng nguyên lý **Secure by Design (An toàn từ Thiết kế)** ngay tại tầng kiến trúc thay vì thiết kế rồi mới vá lỗi:

1. **Thay thế bộ đệm stack cố định bằng bộ đệm heap có giới hạn động:** Thay vì khai báo mảng tĩnh `char stack_buf[16]`, parser nên cấp phát bộ đệm đúng bằng `len + 1` trên Heap hoặc sử dụng cấu trúc chuỗi an toàn kiểu `Bounded String Slice` có mang theo trường độ dài tối đa (`capacity`).

2. **Loại bỏ hoàn toàn hàm không an toàn `memcpy` thô:** Sử dụng các giao diện API an toàn như `memcpy_s` (C11 Bounds Checking Interfaces) hoặc hàm sao chép có kiểm tra dung lượng đích tường minh trước khi thực hiện.

3. **Tách biệt tầng xác thực gói tin (Validation Gate) và tầng xử lý nghiệp vụ (Business Logic):** Đặt bước kiểm tra độ dài payload ngay tại header parser trước khi chuyển dữ liệu cho bất kỳ hàm con nào, ngăn chặn hoàn toàn việc dữ liệu không an toàn đi sâu vào luồng xử lý.

7.2. Trả lời Câu hỏi Bắt buộc: Lỗ hổng Nghi ngờ Còn Tồn tại?

1. **Lỗ hổng từ chối dịch vụ do tiêu tốn tài nguyên (Algorithmic Complexity DoS / Memory Amplification):** Nếu kẻ tấn công liên tục gửi hàng triệu gói tin nhị phân kích thước cực lớn nhưng hợp lệ về Magic và Checksum, chương trình liên tục gọi `malloc` và `free`, có thể dẫn đến hiện tượng phân mảnh bộ nhớ Heap (Heap Fragmentation) hoặc làm kiệt quệ tài nguyên CPU.

2. **Lỗ hổng chênh lệch thời gian thực thi (Timing Side-channel Attack):** Phép tính Checksum XOR và kiểm tra Magic được thực hiện tuần tự và dừng ngay khi phát hiện byte đầu tiên sai (`short-circuit evaluation`). Điều này tạo ra sự chênh lệch thời gian xử lý nhỏ giữa gói tin sai byte đầu tiên và gói tin sai byte cuối cùng, cho phép kẻ tấn công phân tích thời gian để đoán dần cấu trúc gói tin.

7.3. Trả lời Câu hỏi Bắt buộc: Bài học Phương pháp luận Lớn nhất?

1. **Triết lý "Trust, but Verify" (Tin tưởng nhưng phải Thẩm định):** Không thể khẳng định một chương trình C là an toàn chỉ qua việc quan sát mã nguồn hoặc dựa vào các công cụ phân tích cú pháp tĩnh như Cppcheck. Một đoạn mã trông rất gọn gàng vẫn có thể tiềm ẩn lỗi tràn bộ đệm nghiêm trọng khi tiếp nhận dữ liệu bẩn từ người dùng.

2. **Sự kết hợp giữa Kiểm chứng Hình thức (Formal Verification) và Kiểm thử Động (Dynamic Fuzzing):**

- Kiểm chứng hình thức (Model Checking Kripke/CTL, SMT Solver) cho phép ta chứng minh tính đúng đắn trên không gian toán học trừu tượng.
- Nhưng chỉ có kiểm thử mờ động kết hợp công cụ giám sát ô nhớ (AddressSanitizer) mới có thể bắt được các hành vi vi phạm bộ nhớ thực tế trên kiến trúc phần cứng.

7.4. Phân tích Hiện tượng Bỏ lọt Lỗi (False Negative) của Cppcheck

Khi quét toàn diện với cờ khắt khe nhất:

```
cppcheck --enable=all --inconclusive --std=c99 source/fzp1_parser_v0.c
```

Kết quả trả về: **0 cảnh báo, 0 lỗi tràn bộ đệm (0 findings)**.

- **Nguyên nhân kỹ thuật:** Cppcheck phân tích dựa trên cây cú pháp trừu tượng (AST) nội hàm. Trong `fzp1_parser_v0.c`, tham số `len` nhận giá trị từ biến `data[4]` đọc từ luồng tệp nhị phân tại thời điểm chạy (runtime taint). Cppcheck không thể suy diễn được giá trị này trong giai đoạn biên dịch nên cho rằng `len` là hợp lệ.
- **Bài học an toàn:** Phân tích tĩnh là tuyến phòng thủ ban đầu nhưng có điểm mù lớn với dữ liệu động. Bắt buộc phải kết hợp AddressSanitizer và Fuzzing để bảo vệ tầng biên nhị phân.

7.5. Rào cản Toán học của Checksum đối với Black-box Fuzzing

- Trong 15,000 gói tin ngẫu nhiên của Black-box fuzzer, số gói vượt qua kiểm tra Magic Number là 0.
- Kể cả khi may mắn đoán trúng 4 byte `"FZP1"`, xác suất để byte cuối cùng thỏa mãn phương trình XOR vẫn là 1/256.
- Điều này chứng minh định lượng rằng đối với các định dạng nhị phân có ràng buộc bảo vệ tính toàn vẹn (Integrity Check), việc kiểm thử ngẫu nhiên mù quáng là hoàn toàn bất khả thi.

7.6. Sức mạnh Đột phá và Chi phí Tính toán của White-box Concolic SMT Z3

- **Ưu điểm vượt trội:** Chuyển hóa toàn bộ câu lệnh điều kiện thành hệ phương trình logic Bit-Vector. SMT Solver giải quyết trực tiếp bài toán tìm nghiệm trong ~3.9 ms mà không phụ thuộc vào may rủi.
- **Hạn chế và Chi phí (Path Explosion & Non-linear Arithmetic):** Nếu hàm mục tiêu chứa các vòng lặp phụ thuộc dữ liệu phức tạp hoặc hàm băm mật mã (SHA-256), bài toán sẽ trở thành phi tuyến, khiến bộ giải SMT bùng nổ không gian đường đi (Path Explosion). Do đó, phương pháp tiếp cận tối ưu trong công nghiệp là **Hybrid Fuzzing** (kết hợp Greybox tốc độ cao với Concolic giải rào cản).

7.7. Thảo luận về Giới hạn Tích hợp Công cụ Giảng viên & Liêm chính Học thuật

Nhằm quán triệt nguyên tắc liêm chính học thuật (Slide 22 tài liệu *Hướng dẫn Báo cáo BTL*), nhóm báo cáo minh bạch về quá trình vận hành công cụ và dữ liệu thực nghiệm:

1. **Rào cản môi trường của bản thực thi `SecLabFramework.exe`:**

- Ứng dụng `SecLabFramework.exe` do bộ môn cung cấp được đóng gói bằng PyInstaller trên nền tảng Python 3.14. Khi vận hành trên Windows console thông qua các lệnh gọi tự động hóa (CLI), tiến trình phát sinh lỗi mã hóa bảng mã ký tự (`UnicodeEncodeError: 'charmap' codec can't encode character '\u0110'`) do console Windows mặc định dùng trang mã `cp1252`.
- Đồng thời, công cụ là phần mềm GUI độc lập, không hỗ trợ giao diện lập trình không đầu (headless batch-mode API) để chạy lặp lại tự động 30 trials benchmark và xuất tệp JSON cấu trúc.

2. **Giải pháp thực nghiệm độc lập và kiểm chứng khách quan:**

- Để thu thập số liệu benchmark 30 trials hoàn toàn chân thực, nhóm đã xây dựng kịch bản thực nghiệm Python `source/ch4_dynafuzz_fzp1.py` áp dụng chuẩn xác mô hình thuật toán `ch4_dynafuzz` (DynaFuzz) và bộ giải hình thức Z3 Bit-Vector.
- Nhóm cũng xây dựng kịch bản kiểm định độc lập `scripts/validate_fzp1_pipeline.py` nhằm trực tiếp nạp tệp đặc tả SSOT `Nhom11_FZP1_Fuzzing_Spec.xlsx`, dựng cấu trúc Kripke, kiểm tra các công thức CTL và giải toàn bộ 5 bài toán tràn số nguyên bằng Z3 Solver. Kết quả được lưu tại `results/logs/pipeline_log.txt`.

3. **Tuân thủ quy chuẩn đặc tả của `SpecVerificationLab`:**

- Trong tệp đặc tả SSOT Excel, sheet `LoaiTruLanNhau` chỉ hỗ trợ tối đa 1 cặp thuộc tính theo đúng tài liệu thiết kế của bộ môn. Nhóm tôn trọng tuyệt đối quy chuẩn này bằng cách thiết lập cặp thuộc tính loại trừ then chốt là `magic_ok` và `checksum_ok` (1 dòng dữ liệu duy nhất), thay vì cố tình nhồi nhét nhiều dòng làm sai lệch cấu trúc tương thích của tool.

. . . . . .

PHẦN 8: PHỤ LỤC

Phụ lục A: Hướng dẫn Vận hành Công cụ `SecLabFramework.exe` & Quy trình Đối soát Độc lập

Nhóm cung cấp đầy đủ tệp đặc tả SSOT và hướng dẫn chi tiết để giảng viên hoặc người đánh giá có thể kiểm tra trực tiếp:

1. Thao tác nạp tệp Đặc tả SSOT trên giao diện `SecLabFramework.exe` (Tab Ch6):

- **Bước 1:** Khởi chạy tệp `TAI_LIEU_VA_YEU_CAU/04_Cong_Cu_Giang_Vien/SecLabFramework.exe`.
- **Bước 2:** Chọn thẻ Tab **Ch6: Spec -> Kripke -> Code -> CWE/CVE**.
- **Bước 3:** Nhấn nút **Chọn file Excel đặc tả**, điều hướng và chọn tệp: `specification/Nhom11_FZP1_Fuzzing_Spec.xlsx`.
- **Bước 4:** Nhấn nút **Kiểm chứng Kripke & Thuộc tính CTL**: Quan sát đồ thị Kripke 4 trạng thái (`st_idle`, `st_magic_ok`, `st_checksum_ok`, `st_conflict`), xác nhận tính chất không có deadlock (deadlock-free) và cảnh báo vi phạm thuộc tính an toàn tại trạng thái xung đột.
- **Bước 5:** Nhấn nút **Sinh mã C/Python từ đặc tả**: Quan sát mã nguồn khuôn mẫu cho các hàm kiểm toán an toàn bộ nhớ.

2. Thao tác chạy Bộ kiểm định Độc lập của Nhóm (`validate_fzp1_pipeline.py`):

Trong trường hợp môi trường dòng lệnh gặp sự cố về bảng mã ký tự hoặc muốn kiểm chứng tự động toàn bộ 10 sheet của đặc tả:

```
python scripts/validate_fzp1_pipeline.py
```

Kịch bản sẽ trực tiếp đối soát:

- Cấu trúc đồ thị Kripke và 5 công thức CTL trong sheet `ThuocTinhAnToan`.
- Cặp loại trừ lẫn nhau `magic_ok` vs `checksum_ok` trong sheet `LoaiTruLanNhau`.
- Chạy thực tế Z3 SMT Bit-Vector solver trên toàn bộ 5 bài toán biên tại sheet `KiemTraSoHoc`.
- Xuất nhật ký thẩm định ra tệp `results/logs/pipeline_log.txt`.

3. Thao tác chạy Chiến dịch Dynamic Fuzzing Thực nghiệm (`ch4_dynafuzz_fzp1.py`):

```
python scripts/run_fzp1_benchmark.py
python scripts/aggregate_results.py
python scripts/generate_charts.py
```

. . . . . .

Phụ lục B: Toàn văn Mã nguồn C Bản Lỗi (`fzp1_parser_v0.c`)

```
/*
 * BTL Nhom 11 – FZP1 Binary Packet Parser (Version 0: Vulnerable)
 * Lo hong an toan bo nho co chu dich: CWE-121 Stack-based Buffer Overflow
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

#define FZP1_MAGIC "FZP1"
#define FZP1_MAGIC_LEN 4
#define FZP1_HEADER_MIN_SIZE 6
#define STACK_BUFFER_SIZE 16

static uint8_t compute_xor_checksum(const uint8_t *data, size_t len) {
```

```
    uint8_t chk = 0;
    for (size_t i = 0; i < len; i++) {
        chk ^= data[i];
    }
    return chk;
}

static int parse_fzp1_payload(const uint8_t *payload, uint8_t len) {
    char stack_buf[STACK_BUFFER_SIZE];
    printf("[INFO] Parsing payload of length %u into stack buffer (size=%d)...\n", len,
STACK_BUFFER_SIZE);
    /* LOI CWE-121: Khong kiem tra len <= 16 */
    memcpy(stack_buf, payload, len);
    if (len < STACK_BUFFER_SIZE) {
        stack_buf[len] = '\0';
    }
    return 0;
}

int process_fzp1_packet(const uint8_t *data, size_t total_size) {
    if (data == NULL || total_size < FZP1_HEADER_MIN_SIZE) return -1;
    if (memcmp(data, FZP1_MAGIC, FZP1_MAGIC_LEN) != 0) return -1;
    uint8_t payload_len = data[4];
    if (total_size != (size_t)payload_len + FZP1_HEADER_MIN_SIZE) return -1;
    const uint8_t *payload_ptr = data + 5;
    uint8_t expected_checksum = data[5 + payload_len];
    if (compute_xor_checksum(payload_ptr, payload_len) != expected_checksum) return -2;
    printf("[INFO] Header & Checksum verified valid! (Payload length: %u bytes)\n",
payload_len);
    return parse_fzp1_payload(payload_ptr, payload_len);
}

int main(int argc, char **argv) {
    if (argc < 2) return 1;
    FILE *fp = fopen(argv[1], "rb");
    if (!fp) return 1;
    fseek(fp, 0, SEEK_END);
    long fsize = ftell(fp);
    fseek(fp, 0, SEEK_SET);
    if (fsize <= 0 || fsize > 1024) { fclose(fp); return 1; }
    uint8_t *file_buf = (uint8_t *)malloc((size_t)fsize);
    if (!file_buf) { fclose(fp); return 1; }
    fread(file_buf, 1, (size_t)fsize, fp);
    fclose(fp);
    printf("[MAIN] Processing FZP1 file '%s' (%zu bytes)...\n", argv[1], (size_t)fsize);
    int result = process_fzp1_packet(file_buf, (size_t)fsize);
    free(file_buf);
    file_buf = NULL;
    return (result == 0) ? 0 : 2;
}
```

. . . . . .

Phụ lục C: Toàn văn Mã nguồn C Bản Vá An toàn (`fzp1_parser_v1.c`)

```
/*
 * BTL Nhom 11 – FZP1 Binary Packet Parser (Version 1: Patched & Safe)
 * Tuan thu: CERT STR31-C (Bound check), CERT MEM31-C & MEM30-C (Safe Heap)
 */
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <stdint.h>

#define FZP1_MAGIC "FZP1"
#define FZP1_MAGIC_LEN 4
#define FZP1_HEADER_MIN_SIZE 6
#define MAX_PAYLOAD_ALLOWED 16
#define STACK_BUFFER_SIZE (MAX_PAYLOAD_ALLOWED + 1)

static uint8_t compute_xor_checksum(const uint8_t *data, size_t len) {
    uint8_t chk = 0;
    for (size_t i = 0; i < len; i++) {
        chk ^= data[i];
    }
    return chk;
}

static int parse_fzp1_payload(const uint8_t *payload, uint8_t len) {
    char stack_buf[STACK_BUFFER_SIZE];
    printf("[INFO] Parsing payload of length %u into stack buffer (max_allowed=%d)...\n",
len, MAX_PAYLOAD_ALLOWED);
    /* TUAN THU CERT STR31-C: Chan tren chat che */
    if (len > MAX_PAYLOAD_ALLOWED) {
        fprintf(stderr, "[SECURITY_REJECT] Payload length %u exceeds maximum allowed
buffer %d. "
                        "(CERT STR31-C enforcement prevented Stack Buffer Overflow CWE-
121)\n",
                len, MAX_PAYLOAD_ALLOWED);
        return -3;
    }
    memcpy(stack_buf, payload, len);
    stack_buf[len] = '\0';
    printf("[SUCCESS] Payload safely parsed and verified (first byte: 0x%02X)\n",
(uint8_t)stack_buf[0]);
    return 0;
}

int process_fzp1_packet(const uint8_t *data, size_t total_size) {
    if (data == NULL || total_size < FZP1_HEADER_MIN_SIZE) return -1;
    if (memcmp(data, FZP1_MAGIC, FZP1_MAGIC_LEN) != 0) return -1;
    uint8_t payload_len = data[4];
    if (total_size != (size_t)payload_len + FZP1_HEADER_MIN_SIZE) return -1;
    const uint8_t *payload_ptr = data + 5;
    uint8_t expected_checksum = data[5 + payload_len];
    if (compute_xor_checksum(payload_ptr, payload_len) != expected_checksum) return -2;
    printf("[INFO] Header & Checksum verified valid! (Payload length: %u bytes)\n",
payload_len);
    return parse_fzp1_payload(payload_ptr, payload_len);
}

int main(int argc, char **argv) {
    if (argc < 2) return 1;
    FILE *fp = fopen(argv[1], "rb");
    if (!fp) return 1;
    fseek(fp, 0, SEEK_END);
    long fsize = ftell(fp);
    fseek(fp, 0, SEEK_SET);
    if (fsize <= 0 || fsize > 1024) { fclose(fp); return 1; }
    uint8_t *file_buf = (uint8_t *)malloc((size_t)fsize);
    if (!file_buf) { fclose(fp); return 1; }
```

```
    fread(file_buf, 1, (size_t)fsize, fp);
    fclose(fp);
    printf("[MAIN] Processing FZP1 file '%s' (%zu bytes)...\n", argv[1], (size_t)fsize);
    int result = process_fzp1_packet(file_buf, (size_t)fsize);
    free(file_buf);
    file_buf = NULL;
    printf("[MAIN] Finished processing with code: %d\n", result);
    return (result == 0) ? 0 : 2;
}
```

. . . . . .

Phụ lục D: Phân công Công việc và Hướng dẫn Cài đặt / Tái lập Kết quả Thực nghiệm

1. Bảng Phân công Công việc Thành viên (Tỷ lệ 50% - 50%):

Thành viên	Trách nhiệm Chính	Các Hạng mục Thực hiện Cụ thể	Đóng góp
Lưu Đức Hiệp (Nhóm trưởng)	Kiến trúc & Kiểm thử Fuzzing	Thiết kế định dạng nhị phân FZP1, lập trình bộ fuzzer Black-box và Greybox, thiết kế thí nghiệm benchmark 30 trials, phân tích dữ liệu thống kê và vẽ biểu đồ.	50%
Hà Nguyễn Trúc Linh (Thành viên)	Mô hình hóa & An toàn Bộ nhớ	Thiết kế tệp đặc tả Excel 10 sheet SSOT, kiểm chứng Kripke/CTL, lập trình bộ White-box Concolic Z3, xây dựng bản vá CERT C, phân tích log AddressSanitizer.	50%

2. Các Lệnh Kiểm tra và Tái lập Kết quả Thực nghiệm:

```
# 1. Chạy bộ kiểm thử tự động toàn diện
pytest -v

# 2. Biên dịch lại mã nguồn C
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v0.c -o
source/fzp1_parser_v0.exe
gcc -Wall -Wextra -O0 -g -fstack-protector-all source/fzp1_parser_v1.c -o
source/fzp1_parser_v1.exe

# 3. Chạy kiểm chứng tệp tấn công
.\source\fzp1_parser_v0.exe crashes\whitebox_s0_crash_0.bin  # Kết quả: CRASH (Exit Code
3221225501)
.\source\fzp1_parser_v1.exe crashes\whitebox_s0_crash_0.bin  # Kết quả: Từ chối an toàn
(Exit Code 2)

# 4. Tái lập thực nghiệm 30 trials benchmark
python scripts/run_fzp1_benchmark.py
python scripts/aggregate_results.py
```

```
python scripts/generate_charts.py

# 5. Thẩm định toàn vẹn dự án
python scripts/verify_project.py
```